ALGORITHM ANALYSIS

How much work does an algorithm really do?

A visual guide to input size, operation counts, and growth



Made with GAMMA

01 · START WITH THE INPUT

Input size is the scale of the problem

Use **n** to describe how much data the algorithm receives. The right measure depends on the problem.

Choose the dimension that actually drives the work.

01

List

items in a list

02

Matrix

dimensions of a matrix

03

Number

digits in a number

04

Graph

vertices and edges

Made with GAMMA

02 · FIND THE REPEATED WORK

Count the operation that dominates the runtime

A basic operation is the repeated action that contributes most to running time.



Compare

Check a key against another key.



Multiply

Multiply two numbers in a repeated calculation.



Divide

Divide during primality testing.



Visit

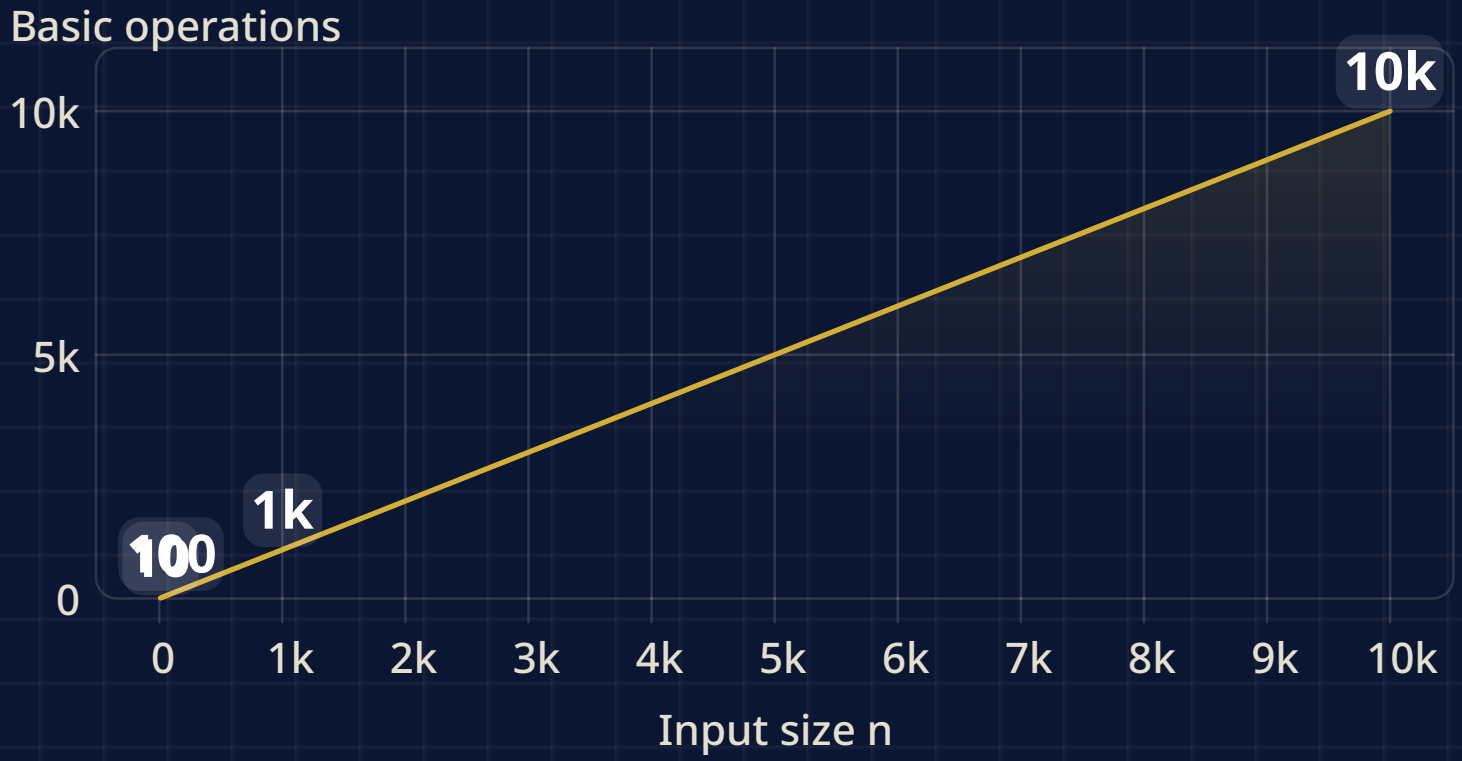
Visit a graph vertex or edge.

Made with GAMMA

03 · TURN COUNTING INTO A FUNCTION

Running time becomes a function of n

Estimate **T(n)** by counting how many times the basic operation executes as input size grows, rather than timing one machine on one input.



THE PATTERN

T(n) = n

The operation count grows in direct proportion to input size.

Made with GAMMA

04 · CHOOSE THE CASE

Worst case asks: what is the most work for size n?

MAXIMUM

Worst

Most work

The maximum cost over all inputs of size n.

MINIMUM

Best

Least work

The minimum cost over all inputs of size n.

DISTRIBUTION

Average

Expected work

Depends on an input distribution.

Worst-case analysis gives a dependable upper bound for every input of that size.

Made with GAMMA

05 · LOOK AT THE SHAPE

Asymptotic growth ignores machine details

FOCUS

Large n

How does the running-time function grow as n becomes large?

REMOVE

Noise

Drop constant factors and lower-order terms when they do not change the growth class.

COMPARE

Shape

Compare algorithms across machines and implementations.

Made with GAMMA

06 · NAME THE BOUND

Big O, Omega, and Theta answer different questions

O

Upper bound

O(g(n)) says the growth is no faster than g(n), up to constant factors.

Ω

Lower bound

Ω(g(n)) says the growth is at least g(n), up to constant factors.

Θ

Tight bound

Θ(g(n)) means the upper and lower bounds match.

Made with GAMMA

07 · SIMPLIFY THE EXPRESSION

Useful rules for simplifying growth

01

Ignore constants

$7n$ is **$\Theta(n)$**. A constant multiplier does not change the growth class.

02

Keep the fastest term

$n^2 + n + 1$ is **$\Theta(n^2)$**.

03

Logs share a class

$\log_2 n$ and **$\log_{10} n$** are both **$\Theta(\log n)$**.

04

Bounds compose

If **f** is **O(g)** and **g** is **O(h)**, then **f** is **O(h)**.

Made with GAMMA

Common running times form a growth ladder

Growth class	Bound	Typical behavior
Logarithmic	**$\Theta(\log n)$**	Repeatedly cut the remaining work, as in binary search.
Linear	**$\Theta(n)$**	Inspect each item once.
Linearithmic	**$\Theta(n \log n)$**	Common in efficient comparison sorting.
Polynomial	**$\Theta(n^2)$, $\Theta(n^3)$**	Nested loops often create these costs.
Exponential	**$\Theta(2^n)$**	Doubling work per added input unit quickly becomes impractical.

Made with GAMMA

Choose the slowest growth you can justify

Use this checklist to justify a bound.

1. **Define n:** State what input size means for this problem.
2. **Identify the basic operation:** Find the repeated action that dominates the runtime.
3. **Count it:** Turn the operation count into a function of n.
4. **Choose the case:** Decide whether you need worst, best, or average case.
5. **Simplify the growth:** Keep the slowest growth you can justify.

For large inputs, $\Theta(\log n)$ and $\Theta(n)$ scale far better than $\Theta(n^2)$. Exponential growth can be useful for small inputs or exhaustive search, but it becomes costly quickly.

Made with GAMMA